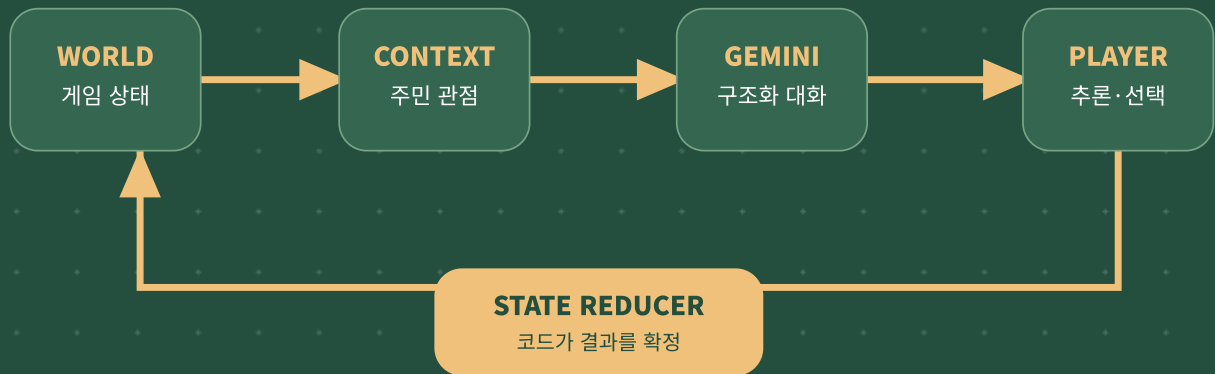


마을의 목소리

AI 활용 기술 문서

구조화된 World State를 주민별 관점으로 변환하고, Gemini가 성격·관계·질문에 맞는 짧은 대화로 전달하는 생활 시뮬레이션 프로토타입



RUNTIME MODEL

Google Gemini 3.6 Flash

CORE CONTRACT

AI는 표현, 코드는 진실

OUTPUT

dialogue · emotion · topic

AI를 “대사 생성기”가 아니라 게임 상태의 번역기로 사용한다

고정 분기 대사는 시설 × 관계 × 최근 사건 × 질문이 늘어날수록 조합 폭발을 일으킵니다. 이 프로젝트는 사실과 결과는 코드에 남기고, 현재 세계를 각 주민다운 말로 표현하는 좁고 검증 가능한 영역에 생성형 AI를 배치했습니다.

01

Authoritative State

시설·행복도·관계도·최근 사건·게임 단계는 TypeScript 객체가 유일한 진실 원천입니다.

02

Perspective Translation

Context Builder가 전체 상태에서 해당 주민에게 필요한 사실과 관계 서술만 추립니다.

03

Bounded Generation

Gemini 출력은 짧은 대사·감정·주제로 제한되고 게임 상태를 변경할 권한이 없습니다.

AI가 담당하는 것

- ✓ 현재 상태를 주민의 성격과 말투로 표현
- ✓ 플레이어의 추천·직접 질문에 응답
- ✓ 관계와 사건을 일상 언어 속에 간접적으로 반영
- ✓ 표정용 감정과 디버그용 주제 반환

AI가 담당하지 않는 것

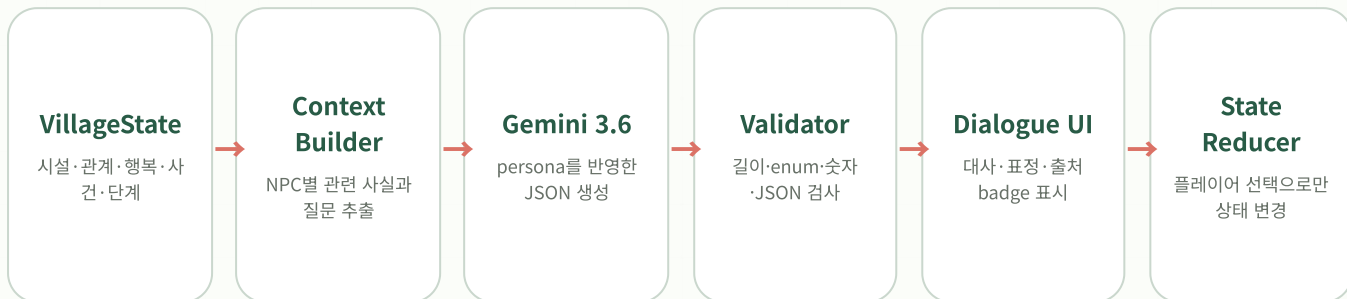
- ✗ 시설의 존재와 건설 결과 결정
- ✗ 행복도·관계도 계산 또는 수정
- ✗ 새로운 사건·과거·약속·인물 창작
- ✗ 개발 회의 해금·결말 판정·저장 처리

설계 결과 생성 응답이 부정확하거나 API가 중단되어도 세계 상태는 오염되지 않습니다. 실패는 “대사 표현 경로”에서만 발생하며, 동일 상태용 검증 대사로 복구됩니다.

기존 방식 대비 필요한 대사 수

방식	새 조건 추가 시	플레이어 질문	세계 일관성
고정 스크립트	모든 조합의 대사를 수동 추가	미리 작성한 선택지만 가능	높지만 확장 비용이 큼
무제한 AI 채팅	확장은 쉽지만 범위가 불명확	자유롭지만 허구 생성 위험	모델 출력에 의존
본 프로젝트	상태와 Context 규칙 추가	40자 직접 질문 + 추천 질문	상태는 코드, 표현만 AI

입력부터 변화까지 이어지는 폐쇄형 플레이 루프



브라우저

src/App.tsx

플레이 단계, 대화·개발·결과
UI

dialogueApi.ts

13초 제한, 응답 검증,
fallback

game/data.ts

주민·시설·단서·검증 대사

game/engine.ts

순수 상태 변경과 해금 조
건

components/*.tsx

코드 기반 동물·아이콘 SVG

styles.css

반응형 마을과 전환 연출

Node.js 서버

server.mjs

정적 빌드 + API proxy

POST /api/dialogue

payload 검사 후 Gemini
호출

GET /api/health

AI 구성 여부와 model 반
환

buildContext()

신뢰 시나리오로 context
생성

callGemini()

서버 전용 키·schema·TTL
cache

sendJson()

no-store 및 보안 header

AI 연결 모드

서버 환경에 키가 있고 Gemini 응답이 schema 검증을 통과하면 source: "ai"로 표시합니다. 같은 주민·단계·시설·질문은 프로세스 메모리 cache에서 재사용합니다.

안전 데모 모드

정적 호스팅, 키 미설정, timeout, HTTP 오류, 파싱·검증 실패 시 game/data.ts의 단계·시설·주민별 대사로 즉시 전환합니다. 게임 로직과 세 결말은 그대로 동작합니다.

UI에 드러나는 생성 출처

상태 표시

AI 연결 또는 안전 데모로 현재 실행
경로를 숨김없이 표시합니다.

대화 badge

검증된 모델 응답은 AI, fallback은
SAFE로 표시합니다.

표정 연동

emotion은 4개 허용값 안에서 SVG
얼굴 연출에만 사용합니다.

대사의 원인이 되는 진실은 모두 코드에 있다

Before

시설 카페만 존재
 행복도 루루 50 · 모카 45 · 두부 60
 루루↔모카 25
 최근 사건 봄비는 카페에서 말다툼
 단계 phase: "before"



After · 공원 예시

시설 카페 + 느티나무 공원
 행복도 루루 65 · 모카 55 · 두부 70
 루루↔모카 50
 최근 사건 공원 벤치에서 오랜만에 대화
 단계 phase: "after"

시설별 결정론적 변화표

시설	행복도 Δ (루루/모카/두부)	관계 Δ (루루-모카 / 루루-두부 / 모카-두부)	코드가 확정하는 최근 사건
공원	+15 / +10 / +10	+25 / +5 / +5	루루와 모카가 공원 벤치에서 대화
오락실	+15 / -5 / +5	-5 / +8 / -2	루루·두부는 게임, 모카는 소음으로 이탈
잡화점	+5 / +5 / +5	0 / 0 / 0	생활은 편리해졌지만 함께 머무는 시간은 동일

상태 불변 조건

- ✓ 행복도와 관계도는 0~100 범위로 clamp
- ✓ before 단계에서 한 번만 시설 선택 허용
- ✓ 이미 선택한 상태에 재호출하면 원본 상태 반환
- ✓ AI 응답은 reducer의 인자가 아님

진행 해금 조건

- ✓ Before에서 주민 2명 대화 → 개발 회의 활성화
- ✓ 시설 선택 → after 단계 및 사건 교체
- ✓ After에서 주민 2명 대화 → 결과 화면 활성화
- ✓ 리셋 시 초기 상태 팩토리로 완전 복원

왜 수치를 숨기는가? 내부 수치는 재현 가능한 규칙과 테스트를 위해 존재합니다. 플레이어에게는 대사·표정·시설 등장·NPC 위치를 통해 같은 변화를 체감시키며, 이것이 “대화가 인터페이스”라는 게임 규칙을 유지합니다.

World State 전체가 아니라 주민이 말할 수 있는 사실만 전달

브라우저 요청

주민 ID · 플레이어 질문 · 단계 · 선택 시설만 힌트로 전달

신뢰
필터

모델 Context

서버가 확정된 시설·사건·관계 표현 + 역할·성격·말투·욕구·60자 이하 질문

루루

활발·사교적·솔직 / 밝고 적극적인 반말 / 친구들과 몸을 움켜쥐어 놀고 싶음

모카

무뚝뚝·자존심·섬세 / 짧고 시니컬한 반말 / 부딪히지 않고 쉬고 싶음

두부

다정·눈치·중재 / 부드럽고 조심스러운 반말 / 세 주민의 자연스러운 화합

Context Builder 핵심 로직

```
profile = residentProfiles[payload.residentId]

scenario = phase === "before"
  ? trustedScenario.before
  : trustedScenario[selectedFacility]

existingFacilities = scenario.facilities
recentEvents = scenario.events
relationships = scenario → 자연어 표현
playerQuestion = String(question).slice(0, 60)

// 브라우저가 보낸 관계·사건 값은 사용하지 않음
```

모카에게 전달되는 예시

```
{
  "role": "고양이 주민 모카",
  "personality": ["무뚝뚝함", "자존심이 강함", "섬세함"],
  "speechStyle": "짧고 시니컬한 반말...",
  "personalDesire": "...편하게 쉬고 싶다.",
  "phase": "before",
  "existingFacilities": ["cafe"],
  "relationships": {
    "lulu": "사이가 좋지 않음",
    "moka": "본인",
    "dubu": "평범함"
  },
  "recentEvents": ["루루와 ... 말다툼했다."],
  "playerQuestion": "루루랑 무슨 일 있었어?"
}
```

포함하지 않는 정보

- × API key와 서버 환경 정보
- × 컴포넌트 내부 상태·UI 위치
- × 모델이 결정할 수 없는 효과 규칙
- × 존재하지 않는 인물·장소·장기 기억

질문·상태 경계

- ✓ UI 40자, 서버 질문 최대 60자
- ✓ 전체 body는 16,384자 초과 시 거부
- ✓ 주민·단계·선택 시설 allowlist 검사
- ✓ 시설·관계·사건은 서버의 신뢰 시나리오로 재구성

관점의 의미 동일한 “공원 없음”도 루루에게는 함께 놀 장소의 부재, 모카에게는 조용히 쉴 장소의 부재, 두부에게는 둘이 자연스럽게 만날 계기의 부재로 표현됩니다.

실제 런타임 프롬프트

프롬프트는 “사실의 경계”, “역할의 경계”, “표현의 길이”, “출력 형식”을 분리해 명시합니다. 아래 내용은 `server.mjs`의 공통 System Instruction 전문입니다.

너는 생활 시뮬레이션 게임의 NPC 대사 작가다.
 게임이 제공한 사실만 사용하고 새로운 시설, 사건, 관계, 약속, 과거를 만들지 마라.
 관계도와 행복도 숫자를 직접 말하지 마라.
 상태를 설명하는 안내자가 아니라 지정된 주민 자신으로만 말하라.
 성격과 관계를 자연스럽게 반영하고 모든 답을 노골적으로 알려주지 마라.
 플레이어 질문에 답하되 한국어 반말 2~3문장, 최대 120자로 짧게 말하라.
 출력은 제공된 JSON schema만 따른다.

User 메시지 조립

다음 JSON은 게임 코드가 제공한 사실이다. 이 주민으로 질문에 답하라.
`{buildContext(payload)가 생성한 JSON}`

System 규칙과 분리된 캐릭터 데이터

주민	Personality	Speech style	Personal desire
루루	활발함, 사교적, 솔직함	밝고 적극적인 반말. 느낌표를 가끔 사용	친구들과 몸을 움직이며 함께 놀고 싶다
모카	무뚝뚝함, 자존심이 강함, 섬세함	짧고 시니컬한 반말. 속마음을 애둘러 말함	사람들과 너무 부딪히지 않고 편하게 쉬고 싶다
두부	다정함, 눈치가 빠름, 중재자	부드럽고 조심스러운 반말. 다른 주민의 마음을 살핌	세 주민이 다시 자연스럽게 어울리길 바란다

입력 예시

모카 / before / “루루랑 무슨 일 있었어?”

- ✓ 카페만 존재
- ✓ 루루와 사이가 좋지 않음
- ✓ 붐비는 카페에서 최근 말다툼

허용되는 응답 예시

```
{
  "dialogue": "별일 아니야. 카페에선 좀 조용히 있고 싶었을 뿐이야.",
  "emotion": "annoyed",
  "topic": "relationship_lulu_moka"
}
```

의도 “공원을 지어라”는 정답 문장을 만들지 않고, 주민마다 다른 욕구가 겹치는 지점을 플레이어가 발견하도록 정보의 직접성을 제한합니다.

모델의 문장을 게임이 신뢰할 수 있는 세 필드로 제한

요청

```
POST /v1beta/models/
  gemini-3.6-flash:generateContent

headers:
  content-type: application/json
  x-goog-api-key: [서버 환경 변수]

generationConfig:
  responseFormat.text.mimeType:
    "APPLICATION_JSON"
  responseFormat.text.schema:
    responseSchema
```

응답 Schema

```
{
  "type": "object",
  "required": [
    "dialogue", "emotion", "topic"
  ],
  "additionalProperties": false,
  "properties": {
    "dialogue": { "type": "string" },
    "emotion": { "enum": [
      "neutral", "happy",
      "annoyed", "worried"
    ] },
    "topic": { "enum": [7개 허용 주제] }
  }
}
```

JSON 파싱

일반 JSON 우선, 필요 시 첫 중괄호
구간 재파싱

대사 길이

생성 목표 120자, 서버·클라이언트
상한 180자

Enum 검사

감정 4개, 주제 7개 외 값 거부

숫자 차단

1~3자리 숫자가 대사에 있으면 거부

허용 Topic과 사용 목적

Topic	의미	게임에서의 용도
shared_space	함께 머무는 공간	Before 대사의 주제 추적 및 디버그
quiet_space	조용히 쉬는 공간	
relationship_lulu_moka	루루-모카 관계	
park / arcade / shop	선택된 시설	After 상태 반영 여부 확인
village_change	마을 전체 변화	

시간과 cache

- ✓ 서버 호출 12초, 클라이언트 13초 후 Abort
- ✓ 주민·단계·시설·질문 조합을 메모리 cache
- ✓ 10분 TTL · 최대 120개 항목
- ✓ 서버 프로세스 종료 시 cache 초기화

출력의 권한

- ✓ dialogue → 말풍선 텍스트
- ✓ emotion → 캐릭터 표정만 변경
- ✓ topic → 상태 반영 점검
- ✓ 세 필드 모두 reducer 호출 권한 없음

이중 검증 서버가 먼저 결과를 검사하고, 브라우저가 다시 객체 형태·길이·enum을 확인합니다. 어느 단계에서든 실패하면 동일한 상태용 fallback 대사를 사용합니다.

AI 실패가 시연 실패가 되지 않도록 설계

NPC 클릭·질문

API 요청

HTTP-JSON-schema 검사

성공: AI 대사
실패: 검증 대사

동일한 게임 진행

신뢰성 장치

- ✓ 주민 × Before/After × 시설별 fallback 완비
- ✓ API health를 1.8초 내 확인해 실행 경로 표시
- ✓ AbortController 기반 12/13초 이중 timeout
- ✓ 응답 cache로 같은 질문의 지연·비용 감소
- ✓ 시설 선택과 결과 계산은 네트워크와 독립

서버 입력·응답 방어

- ✓ 주민·단계·선택 시설 allowlist 검사
- ✓ 관계·사건·시설은 서버 신뢰 시나리오로 재구성
- ✓ IP당 12회/분 · 전체 240회/시간 · AI 동시 4건
- ✓ origin 검사, body·질문 길이 상한
- ✓ no-store·nosniff·CSP header

API key 보안

구현 원칙

키는 GEMINI_API_KEY 서버 환경 변수에서만 읽습니다. VITE_ 접두사를 사용하거나 클라이언트 번들·소스 저장소·문서에 값을 기록하지 않습니다.

정적 배포의 경계

GitHub Pages에는 비밀 저장소와 서버 실행 환경이 없습니다. 따라서 공개 정적 빌드는 안전 대사 경로를 유지하며, live AI는 키를 보관하는 Node 서버 환경에서 실행합니다.

개인정보와 데이터

항목	처리
직접 질문	40자 이내 텍스트를 현재 상태와 함께 Gemini API에 전송할 수 있음. 게임은 개인정보 입력을 요구하지 않음
계정·저장	로그인, 사용자 식별자, 저장/불러오기, 분석 SDK를 구현하지 않음
서버 기록	애플리케이션 코드에서 질문·응답을 영구 저장하지 않음. debug 출력은 명시적 환경 설정에서만 동작
Cache	프로세스 메모리 안에서만 유지되며 재시작 시 사라짐

알려진 한계와 다음 단계

정성적 검증

허구를 완전히 증명하는 의미 검증은 없으므로, 사실 범위를 줄이고 fallback으로 위험을 제한했습니다.

단기 기억

현재 질문과 World State만 사용합니다. 긴 대화 이력이나 장기 기억은 프로토타입 범위 밖입니다.

운영 강화

현재 origin·분당·시간당·동시 호출 제한을 적용했습니다. 장기 운영 시 관리형 서버리스와 일일 예산 경보·주기적 key rotation을 추가합니다.

활용 내역·출처·검증 기록

런타임 AI

Google Gemini 3.6 Flash

NPC의 현재 상태 기반 대사와 emotion, topic을 구조화 JSON으로 생성합니다. 시설 효과·관계 변경·결말 판정에는 관여하지 않습니다.

주요 입력 주민 persona, 존재 시설, 관계 요약, 최근 사건, 단계, 질문
인간 설계 세계 데이터, 효과표, 프롬프트 경계, schema, fallback 대사

개발 보조 AI

OpenAI Codex

사용자 작성 기획서를 기반으로 프로토타입 구조 검토, React/TypeScript 구현 보조, 프롬프트·검증 로직 점검, 자동 테스트와 제출 문서 초안 작성에 사용했습니다.

검수 방식 생성 결과를 저장소의 실제 코드와 대조하고 TypeScript build, 단위 테스트, 브라우저 smoke 시나리오로 확인했습니다. 최종 선택과 제출 책임은 참가자에게 있습니다.

외부 에셋 및 오픈소스

항목	버전 / 라이선스	용도 및 출처
React / React DOM	19.2.8 · MIT	UI 상태·렌더링 · github.com/facebook/react
Vite / plugin-react	8.2.1 / 6.0.5 · MIT	개발·빌드 · github.com/vitejs/vite
TypeScript	7.0.2 · Apache-2.0	정적 타입 · github.com/microsoft/TypeScript
Vitest	4.1.10 · MIT	상태 reducer 단위 테스트 · github.com/vitest-dev/vitest
Playwright Core	1.62.1 · Apache-2.0	브라우저 smoke · github.com/microsoft/playwright
Noto Sans/Serif KR	SIL OFL 1.1	문서 및 시스템 폰트 fallback · github.com/notofonts/noto-cjk

시각·음향 에셋 고지 외부 이미지, 외부 아이콘 파일, BGM, 효과음을 사용하지 않았습니다. 동물 캐릭터·아이콘·마을 건물은 프로젝트 내부의 React 인라인 SVG와 HTML/CSS 도형으로 직접 구성했습니다. 문서 도식 역시 CSS와 인라인 SVG만 사용했습니다.

검증 기록

- ✓ Vitest reducer 테스트 4/4 통과
- ✓ Before→공원→After E2E 경로 스크립트
- ✓ 세 시설의 서로 다른 관계 결과 검사
- ✓ live AI badge를 확인하는 별도 smoke 스크립트
- ✓ 브라우저 캡처 5종 생성

재현 명령

```
npm ci
npm run check

# 안전 대사 포함 통합 서버
npm start

# live AI는 서버 환경에
# GEMINI_API_KEY를 별도 설정
```

공식 기술 참고

- Gemini API key 및 보안: ai.google.dev/gemini-api/docs/api-key
- Structured outputs: ai.google.dev/gemini-api/docs/generate-content/structured-output
- GitHub Pages 정적 호스팅: docs.github.com/en/pages/.../what-is-github-pages
- 소스 저장소: github.com/jinjinjin459/nan2026-village-voices